

Lecture 3: Solving and modeling LPs

Fall 2025

Prof. Udell

1 Solving linear programs

We now turn to algorithms for solving linear programs. The main idea, motivated by the geometry we have developed, is that if an LP has a solution, then some *corner point* (BFS / vertex / extreme point) is optimal. Thus algorithms can search among vertices rather than the entire feasible polyhedron.

1.1 Algorithms for LPs

- *Enumerate vertices*: generate all BFS and evaluate the objective.
- *Fourier–Motzkin elimination*: eliminate variables to project down to a feasible interval.
- *Simplex method*: walk along edges from BFS to BFS, improving the objective.
- *Ellipsoid method*: first polynomial-time algorithm, theoretical but not practical.
- *Interior point methods*: polynomial-time and practically efficient.
- *First-order methods*: scalable to very large problems, using gradient information only.

Remark 1.1. Enumeration and elimination are conceptually simple but not practical. The simplex algorithm remains a workhorse in practice. Interior point methods (IPMs) provide an alternative with strong complexity guarantees. First-order methods are attractive for large-scale machine learning problems.

Fourier–Motzkin elimination

Fourier–Motzkin elimination is a procedure for projecting a polyhedron onto a subspace by eliminating variables. It is a conceptually simple algorithm, and one of the earliest algorithms for linear programming, but it is not practical for large problems. The algorithm is named after Joseph Fourier, who proposed the method in 1826, and Theodore Motzkin who re-discovered it in 1936. As a side benefit, it shows that the projection of a polyhedron is again a (closed) polyhedron.

We illustrate the method with an example.

Example 1.2 (Fourier–Motzkin elimination). Consider the system

$$\begin{aligned}x_1 + 2x_2 &\leq 4, \\ -x_1 + x_2 &\leq 1, \\ x_1, x_2 &\geq 0.\end{aligned}$$

We eliminate x_1 . Collect inequalities bounding x_1 :

$$\{0, x_2 - 1\} \leq x_1 \leq 4 - 2x_2.$$

Hence feasible x_2 must satisfy

$$0 \leq 4 - 2x_2, \quad x_2 - 1 \leq 4 - 2x_2, \quad x_2 \geq 0.$$

Simplifying, $x_2 \in [0, 5/3]$. Thus the projection of the feasible region onto x_2 is the interval $[0, 5/3]$. This method shows that the projection of a polyhedron is again a (closed) polyhedron.

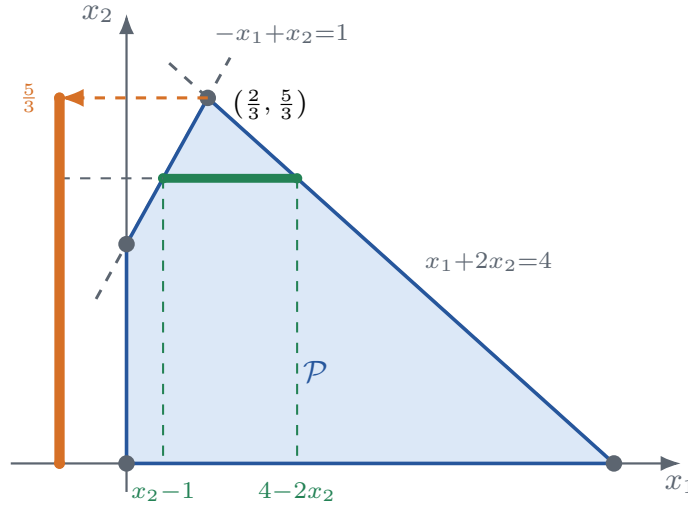


Figure 1: Fourier–Motzkin elimination projects the feasible polygon onto the x_2 axis. The projected interval is exactly $[0, 5/3]$.

1.2 Enumerating vertices

One of the main benefits of the basic feasible solution (BFS) concept is that it allows us to enumerate all vertices of a polyhedron.

Proposition 1.3. *Let $A \in \mathbb{R}^{m \times n}$ with full row rank m . Then every BFS corresponds to some choice of m columns A_S with $|S| = m$ and A_S invertible. The BFS is*

$$x_S = A_S^{-1}b, \quad x_{\bar{S}} = 0.$$

If $x_S \geq 0$, this point is feasible.

Proof. Direct from the definition of a BFS: pick m basic variables, solve the equality, and set the rest to zero. Feasibility requires nonnegativity. Thus every BFS arises in this way. $\square$

Remark 1.4. Enumerating all BFS requires considering $\binom{n}{m}$ bases, which is exponential in m . Hence naive enumeration is not a practical algorithm.

1.3 Simplex algorithm

Consider first the two-variable example below. Starting at $x^{(0)} = (0, 0)$, we increase x_2 until the second constraint binds at $x^{(1)} = (0, 2)$. We then move along the edge to $x^{(2)} = (3, 1)$. The objective values are 0, -4 , and -5 , respectively.

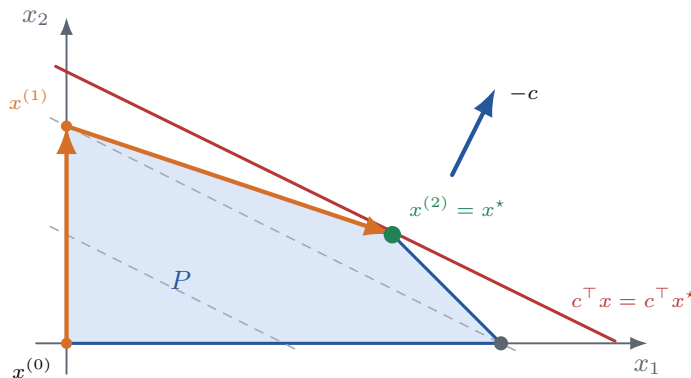


Figure 2: A simplex walk from $x^{(0)}$ to the optimal vertex $x^{(2)}$.

Abstracting from the example, the simplex algorithm performs local search on the set of BFS.

Definition 1.5 (Simplex algorithm). The simplex algorithm is a *local search* on BFS:

1. Start at some BFS x .
2. Move to a neighboring BFS x' with strictly better objective $c^T x'$.
3. Repeat until no improving neighbor exists.

Historically, the simplex algorithm was the first practical method for solving LPs, developed by George Dantzig in the 1940s.

The simplex algorithm raises several key questions:

- How to find an initial BFS?
- How to choose a neighboring BFS that improves the objective?
- How to prove optimality when no improving neighbor exists?

1.4 Finding an initial BFS

One approach is to solve an auxiliary problem for which a BFS is obvious:

$$\begin{aligned} & \text{minimize} && \sum_{i=1}^m z_i \\ & \text{subject to} && Ax + Dz = b, \\ & && x, z \geq 0, \end{aligned}$$

where $D \in \mathbb{R}^{m \times m}$ is diagonal with $D_{ii} = \text{sign}(b_i)$. The BFS $x = 0, z = |b|$ is always feasible. If the optimal solution has $z = 0$, then x is a BFS for the original problem.

Exercise 1.6. Show that the auxiliary problem has the following properties:

1. If the original problem is feasible, then the auxiliary problem has optimal value zero.
2. If the original problem is infeasible, then the auxiliary problem has optimal value greater than zero.
3. Any feasible solution of the auxiliary problem with $z = 0$ gives a feasible solution of the original problem.

1.5 Optimization via reduced costs

Let us now address the other two questions: how to choose an improving neighboring BFS, and how to prove optimality when no improving neighbor exists. We will use the concept of *reduced cost* to answer both questions: the reduced cost tells us how the objective changes when we move from one BFS to a neighboring BFS.

Suppose x is a nondegenerate BFS with active set S . Consider entering variable $j \notin S$. We will increase x_j from zero, and adjust the basic variables x_S to maintain feasibility. Define the *basic direction* d^j :

$$d_j^j = 1, \quad d_S^j = -A_S^{-1}A_j, \quad d_i^j = 0 \text{ for } i \notin S \cup \{j\}.$$

Then $Ad^j = 0$, so feasibility is preserved. For sufficiently small $\theta > 0$, $x + \theta d^j \geq 0$. The change in the objective is

$$c^T(x + \theta d^j) - c^T x = \theta \bar{c}_j,$$

where

$$\bar{c}_j := c_j - c_S^T A_S^{-1} A_j$$

is the *reduced cost* of variable j .

An improving neighbor. To find a neighboring BFS with better objective, we can compute the reduced costs $\bar{c}_j$ for all $j \notin S$. If any $\bar{c}_j < 0$, then moving in direction d^j improves the objective. We can increase θ until some basic variable x_i hits zero, which gives a new BFS with active set $S \cup \{j\} \setminus \{i\}$. This is called a *pivot* operation.

Remark 1.7. The choice of entering variable j and leaving variable i can affect the number of iterations required for convergence. There are many pivot rules; a common choice is to pick the entering variable with most negative reduced cost.

An optimality criterion. The reduced cost also gives a simple optimality test:

Proposition 1.8 (Reduced cost optimality test). *If all reduced costs $\bar{c}_j \geq 0$ for $j \notin S$, then the BFS x is optimal.*

Proof. Let x' be any feasible point. Since

$$A_S x_S = b, \quad A_S x'_S + A_{\bar{S}} x'_{\bar{S}} = b,$$

we have

$$x'_S = x_S - A_S^{-1} A_{\bar{S}} x'_{\bar{S}}.$$

Substituting into the objective gives

$$\begin{aligned} c^T x' &= c_S^T x'_S + c_{\bar{S}}^T x'_{\bar{S}} \\ &= c_S^T x_S + (c_{\bar{S}}^T - c_S^T A_S^{-1} A_{\bar{S}}) x'_{\bar{S}} \\ &= c^T x + \bar{c}_{\bar{S}}^T x'_{\bar{S}} \\ &\geq c^T x, \end{aligned}$$

because $\bar{c}_{\bar{S}} \geq 0$ and $x'_{\bar{S}} \geq 0$. Thus no feasible point has a smaller objective value than x , so x is optimal. $\square$

Proposition 1.9. *The feasible set of the LP lies in the cone generated by the basic directions $x + \text{cone}(\{d^j : j \notin S\})$ for any BFS x .*

Proof. The basic directions d^j for $j \notin S$ form a basis for the nullspace of A . To see this, note that $Ad^j = 0$ by construction. Moreover, the $n-m$ vectors d^j are linearly independent: if $\sum_{j \notin S} \alpha_j d^j = 0$, then looking at the components in $\bar{S}$ shows $\alpha_j = 0$ for all $j \notin S$. Since $\dim(\text{null}(A)) = n - m$, the d^j form a basis.

Now consider any feasible point x' with $Ax' = b$ and $x' \geq 0$. Then $d = x' - x$ satisfies $Ad = 0$, so d lies in the nullspace of A . Thus $d = \sum_{j \notin S} \alpha_j d^j$ for some $\alpha_j \in \mathbb{R}$. Since $x' = x + d \geq 0$ and $x_j = 0$ for all $j \notin S$, we must have $\alpha_j \geq 0$ for all $j \notin S$. Thus $x' \in x + \text{cone}(\{d^j : j \notin S\})$. $\square$

Gotcha 1.10. The condition $\bar{c}_j \geq 0$ for every nonbasic variable is sufficient for optimality even when the BFS is degenerate. Nondegeneracy matters for the converse local step: if x is degenerate, a negative reduced cost can point in a direction whose maximum feasible step is zero. Pivot selection and cycling therefore require extra care at degenerate BFS.

1.6 Worked example

Example 1.11 (Two-variable LP). Consider

$$\begin{aligned} &\text{minimize} && -x_1 - 2x_2 \\ &\text{subject to} && x_1 + x_2 \leq 4, \\ & && x_1 + 3x_2 \leq 6, \\ & && x_1, x_2 \geq 0. \end{aligned}$$

The feasible region is a polygon in $\mathbb{R}^2$ with BFS at $(0, 0)$, $(0, 2)$, $(3, 1)$, $(4, 0)$. Evaluating the objective shows the minimum occurs at $(3, 1)$. Reduced costs confirm optimality: no improving direction exists at this BFS.

1.7 Exercises

Exercise 1.12. Consider the LP

$$\begin{array}{ll} \text{minimize} & x_1 - x_2 \\ \text{subject to} & x_1 + 2x_2 \leq 4, \\ & 2x_1 + x_2 \leq 5, \\ & x_1, x_2 \geq 0. \end{array}$$

- (a) List all BFS by choosing pairs of active constraints.
- (b) For each BFS, compute the objective value.
- (c) Identify the optimal BFS.
- (d) Verify optimality using reduced costs.

2 Modeling linear and mixed-integer programs

Modeling is the process of describing exactly which decisions are allowed and which allowed decision is best. A useful workflow is:

1. define each decision variable and its bounds;
2. translate one requirement at a time;
3. test the formulation on boundary cases.

Three recurring techniques are to introduce auxiliary variables, use binary variables for yes/no decisions, and use variable bounds to activate or deactivate constraints.

2.1 Two standard-form reformulations

An LP in standard form has equality constraints and nonnegative variables. Inequalities and free variables can be converted to this form by direct substitution.

Definition 2.1 (Slack variables). For any vector a and scalar b ,

$$a^T x \leq b \iff a^T x + s = b, \quad s \geq 0.$$

The variable s measures the unused amount of the resource represented by the inequality.

Definition 2.2 (Splitting a free variable). A free variable $x_i \in \mathbb{R}$ can be expressed as

$$x_i = x_i^+ - x_i^-, \quad x_i^+, x_i^- \geq 0.$$

The substitution must be made everywhere, including the objective. For example, $c_i x_i$ becomes $c_i x_i^+ - c_i x_i^-$.

Example 2.3 (Conversion to standard form). Consider

$$\begin{array}{ll} \text{minimize} & -2x_1 + x_2 \\ \text{subject to} & 2x_1 + 3x_2 \leq 5, \quad x_1 \in \mathbb{R}, \quad x_2 \geq 0. \end{array}$$

Introduce $x_1^+, x_1^-, s \geq 0$. The standard-form problem is

$$\begin{array}{ll} \text{minimize} & -2x_1^+ + 2x_1^- + x_2 \\ \text{subject to} & 2x_1^+ - 2x_1^- + 3x_2 + s = 5, \\ & x_1^+, x_1^-, x_2, s \geq 0. \end{array}$$

2.2 The epigraph trick

Definition 2.4 (Epigraph). The *epigraph* of a function f is the set of points above its graph:

$$\text{epi } f = \{(x, t) : f(x) \leq t\}.$$

Minimizing over the epigraph makes the upper bound tight. For a feasible set $\mathcal{X}$,

$$\begin{array}{ll} \text{minimize}_{x \in \mathcal{X}} & f(x) \\ \iff & \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & x \in \mathcal{X}, \\ & f(x) \leq t. \end{array} \end{array}$$

This reformulation is useful when the epigraph constraint has a linear description.

Absolute values. For each component, $|x_i| \leq t_i$ is equivalent to $-t_i \leq x_i \leq t_i$. Hence

$$\begin{array}{ll} \text{minimize}_{x \in \mathcal{X}} & \sum_{i=1}^n |x_i| \\ \iff & \begin{array}{ll} \text{minimize}_{x,t} & \mathbf{1}^T t \\ \text{subject to} & x \in \mathcal{X}, \\ & -t \leq x \leq t. \end{array} \end{array}$$

At an optimum, $t_i = |x_i|$. The inequalities alone do not force equality; the minimizing objective does.

Pointwise maxima. If each f_i is affine, then

$$\begin{array}{ll} \text{minimize}_{x \in \mathcal{X}} & \max_{i=1, \dots, m} f_i(x) \\ \iff & \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & x \in \mathcal{X}, \\ & f_i(x) \leq t, \quad i = 1, \dots, m. \end{array} \end{array}$$

The right-hand problem is an LP when $\mathcal{X}$ has a linear description.

Example 2.5 (Worst-case residual). The problem

$$\underset{x}{\text{minimize}} \quad \max_{i=1,\dots,m} |a_i^T x - b_i|$$

has the LP formulation

$$\begin{aligned} & \underset{x,t}{\text{minimize}} \quad t \\ & \text{subject to} \quad -t \leq a_i^T x - b_i \leq t, \quad i = 1, \dots, m. \end{aligned}$$

Each constraint bounds one absolute residual by t , and the objective finds the smallest uniform bound.

2.3 Binary variables and logical constraints

A binary variable encodes a proposition: value one means true and value zero means false. Introducing binary variables changes an LP into a mixed-integer linear program (MILP).

Let $z_A, z_B \in \{0, 1\}$ indicate whether projects A and B are funded. The implication “if A , then B ” is equivalent to

$$z_A \leq z_B.$$

Indeed, this inequality excludes exactly the forbidden assignment $(z_A, z_B) = (1, 0)$. The expression $1 - z_A$ encodes “not A .”

Sums of binary variables count how many propositions are true:

$$\begin{aligned} \text{at least one} & : \sum_i z_i \geq 1, \\ \text{at most one} & : \sum_i z_i \leq 1, \\ \text{exactly } k & : \sum_i z_i = k. \end{aligned}$$

Example 2.6 (Project logic). Suppose project-selection variables are binary. The following logical rules have linear formulations:

$$\begin{aligned} \text{fund } A \text{ only if } E \text{ is funded} & : z_A \leq z_E, \\ \text{if } A \text{ is not funded, then fund } G & : 1 - z_A \leq z_G, \\ \text{fund exactly one of } B \text{ and } C & : z_B + z_C = 1, \\ \text{funding } A \text{ or } B \text{ requires funding } H & : z_A \leq z_H, \quad z_B \leq z_H. \end{aligned}$$

The single inequality $z_A + z_B \leq z_H$ would be too strong: it forbids funding both A and B even when H is funded.

2.4 Conditional linear constraints

Suppose $z \in \{0, 1\}$ and we require

$$z = 1 \implies a^T x \leq b.$$

The constraint

$$a^T x \leq b + M(1 - z)$$

enforces the desired inequality when $z = 1$ and relaxes it when $z = 0$. The constant M should not be chosen merely to be “very large.” If x already belongs to a known set $\mathcal{X}$, the tightest nonnegative valid value is

$$M = \max \left\{ 0, \max_{x \in \mathcal{X}} (a^T x - b) \right\}.$$

Tight, valid bounds generally produce stronger relaxations and more reliable numerical behavior.

Example 2.7 (Classroom capacity). Let $X_{ij} = 1$ indicate that class i uses room j . Suppose enrollment is a decision variable satisfying $0 \leq p_i \leq \bar{p}_i$, and room j has capacity c_j . The implication

$$X_{ij} = 1 \implies p_i \leq c_j$$

can be written as

$$p_i \leq c_j + M_{ij}(1 - X_{ij}), \quad M_{ij} = \max\{0, \bar{p}_i - c_j\}.$$

If enrollment p_i is fixed data, omit assignments for which $p_i > c_j$ instead of introducing a big- M constraint.

Modern solvers also provide indicator constraints that express an implication directly. Good variable bounds remain important even when the modeling language hides the explicit value of M .

2.5 On/off production

Example 2.8 (A plant that can be on or off). The decisions are production $x \geq 0$ and operating status $z \in \{0, 1\}$. The data are

- $0 < L \leq U$, the minimum and maximum production when the plant operates;
- $c \geq 0$, the production cost per unit;
- $f \geq 0$, the fixed cost whenever the plant operates.

The formulation

$$Lz \leq x \leq Uz, \quad \text{cost} = cx + fz$$

has the desired behavior. When $z = 0$, the bounds force $x = 0$. When $z = 1$, they reduce to $L \leq x \leq U$. The previously seen fixed-cost constraint $x \leq Uz$ is therefore a conditional constraint whose constant comes from a known upper bound.

2.6 Equivalence and two implications

An implication constrains one direction only. An equivalence requires both directions. Let $q \in \{0, 1, \dots, U\}$ and suppose we want to define

$$z = 1 \iff q \geq k.$$

The exact linear formulation is

$$q \geq kz, \quad q \leq (k - 1) + (U - k + 1)z.$$

If $z = 0$, the second inequality gives $q \leq k - 1$. If $z = 1$, the first gives $q \geq k$. Integrality makes the strict separation exact.

For a continuous quantity q , the reverse implication would require the strict inequality $q < k$ when $z = 0$. Ordinary LP and MILP constraints use non-strict inequalities. A model must therefore introduce a nonzero tolerance or allow either value of z at the boundary $q = k$.

Example 2.9 (An indicator for a count). Let $x_1, \dots, x_5 \in \{0, 1\}$ denote five project selections, and let $z = 1$ exactly when at least three projects are selected. With $q = \sum_{i=1}^5 x_i$, $U = 5$, and $k = 3$, the formulation becomes

$$\sum_{i=1}^5 x_i \geq 3z, \quad \sum_{i=1}^5 x_i \leq 2 + 3z.$$

The first inequality forces the count to be at least three when $z = 1$. The second forces $z = 1$ once the count reaches three.

2.7 Checks for logical formulations

For each logical constraint:

- identify which assignments should be allowed;
- translate each implication separately;
- substitute $z = 0$ and $z = 1$ to check both cases;
- derive constants from valid bounds rather than choosing arbitrary large values.